Week 8: Linear Networks

CSCI 447: Machine Learning– Fall 2026

Gradient Descent – Logistic Regression

- The classic approach to solve is via gradient descent.
- We seek $\mathbf{w}$ such that for all w_j

$$\frac{\partial \ell(\mathbf{w})}{\partial w_j} = 0$$

- Note that

$$\frac{\partial \ell(\mathbf{w})}{\partial w_j} = \sum_k x_j^k \left(C^k - \hat{P}\left(C^k = 1 \middle| \mathbf{w}\right) \right)$$

- Weight update rule:

$$w_j \leftarrow w_j + \eta \sum_k x_j^k \left(C^k - \hat{P}\left(C^k = 1 \middle| \mathbf{w}\right) \right)$$

Regularization

- Avoiding learning noise, which would lead to overfitting.
- Penalize large weights.

$$\mathbf{w} = \underset{\mathbf{w}}{\operatorname{argmax}} \sum_k \ln P\left(C^k \middle| \mathbf{x}^k, \mathbf{w}\right) - \frac{\lambda}{2}\|\mathbf{w}\|^2$$

- New gradient

$$\frac{\partial \ell(\mathbf{w})}{\partial (w_j)} = \sum_k x_j^k \left(C^k - \hat{P}\left(C^k = 1 \middle| \mathbf{w}\right)\right) - \lambda w_j$$

- New update

$$w_j \leftarrow w_j + \eta \sum_k x_j^k \left(C^k - \hat{P}\left(C^k = 1 \middle| \mathbf{w}\right)\right) - \eta\lambda w_j$$

Two-Class Logistic Regression

For $j = 0, \ldots, d$
  $w_j \leftarrow$ rand(-0.01,0.01)
Repeat
  For $j = 0, \ldots, d$
    $\Delta w_j \leftarrow 0$
  For $t = 1, \ldots, N$
    $o \leftarrow 0$
    For $j = 0, \ldots, d$
      $o \leftarrow o + w_j x_j^t$
    $y \leftarrow \text{sigmoid}(o)$
    $\Delta w_j \leftarrow \Delta w_j + (r^t - y) x_j^t$
  For $j = 0, \ldots, d$
    $w_j \leftarrow w_j + \eta \Delta w_j$
Until convergence

Multi-Class Classification

- Extend the log likelihood function.
- For $i = 1, \ldots, K-1$

$$P(C = c_i|\mathbf{x}) = \frac{\exp(w_0 + \sum_{j=1}^{d} w_j x_j)}{1 + \sum_{c=1}^{K-1} \exp(w_{c0} + \sum_{j=1}^{d} w_j x_j)}$$

- For $i = K$

$$P(C = c_i|\mathbf{x}) = \frac{1}{1 + \sum_{c=1}^{K} \exp(w_{c0} + \sum_{j=1}^{d} w_j x_j)}$$

- Update rule

$$w_{ji} \leftarrow w_{ji} + \eta \sum_{k} x_j^i (\delta(C = c_k) - \hat{P}(C = c_k|\mathbf{x}^i, \mathbf{w})) - \eta\lambda w_{ji}$$

Classification with Softmax

- In logistic regression (and similar models), probabilistic classification is often done via the softmax function (based on the logistic, sigmoid function):

$$y_i = P(C_i|\mathbf{x}) = \frac{\exp\left[\mathbf{w}_i^\top \mathbf{x} + w_{i0}\right]}{\sum_{j=1}^{K} \exp\left[\mathbf{w}_j^\top \mathbf{x} + w_{j0}\right]}.$$

- This assumes we treat *all* of the classes uniformly.
- We assume a multinomial class distribution, $\mathbf{r}^t|\mathbf{x}^t \sim \text{Mult}_k(\mathbf{y}^t)$, where $y_i^t \equiv P(C_i|\mathbf{x}^t)$.

Cross Entropy Loss

- Using softmax, we want to maximize the sample likelihood of the weights given the data is

$$L(\{\mathbf{w}_i, w_{i0}\}|\mathcal{X}) = \prod_t \prod_i \left(y_i^t\right)^{r_i^t}.$$

- To learn the parameters $\mathbf{w}_i^\top$ for each class C_i, we minimize the cross-entropy loss, defined as the negative log of the likelihood:

$$\ell(\{\mathbf{w}_i, w_{i0}\}|\mathcal{X}) = -\sum_t \sum_i r_i^t \log y_i^t$$

Gradient Descent

- We use gradient descent to learn the weights.
- We can show that for $y_i = \exp(a_i) \,/\, \sum_j \exp(a_j)$,

$$\frac{\partial y_i}{\partial a_j} = y_i\left(\delta_{ij} - y_j\right),$$

where

$$\delta_{ij} = \begin{cases} 1 & i = j \\ 0 & i \neq j \end{cases}.$$

Update Rule

- Based on this, we can derive the update rule as follows:

$$
\begin{aligned}
\Delta \mathbf{w}_j &= \eta \sum_t \sum_i \frac{r_i^t}{y_i^t} y_i^t \left(\delta_{ij} - y_j^t\right) \mathbf{x}^t \\
&= \eta \sum_t \sum_i r_i^t \left(\delta_{ij} - y_j^t\right) \mathbf{x}^t \\
&= \eta \sum_t \left[\sum_i r_i^t \delta_{ij} - y_j^t \sum_i r_i^t \right] \mathbf{x}^t \\
&= \eta \sum_t \left(r_j^t - y_j^t\right) \mathbf{x}^t.
\end{aligned}
$$

- For the bias term,

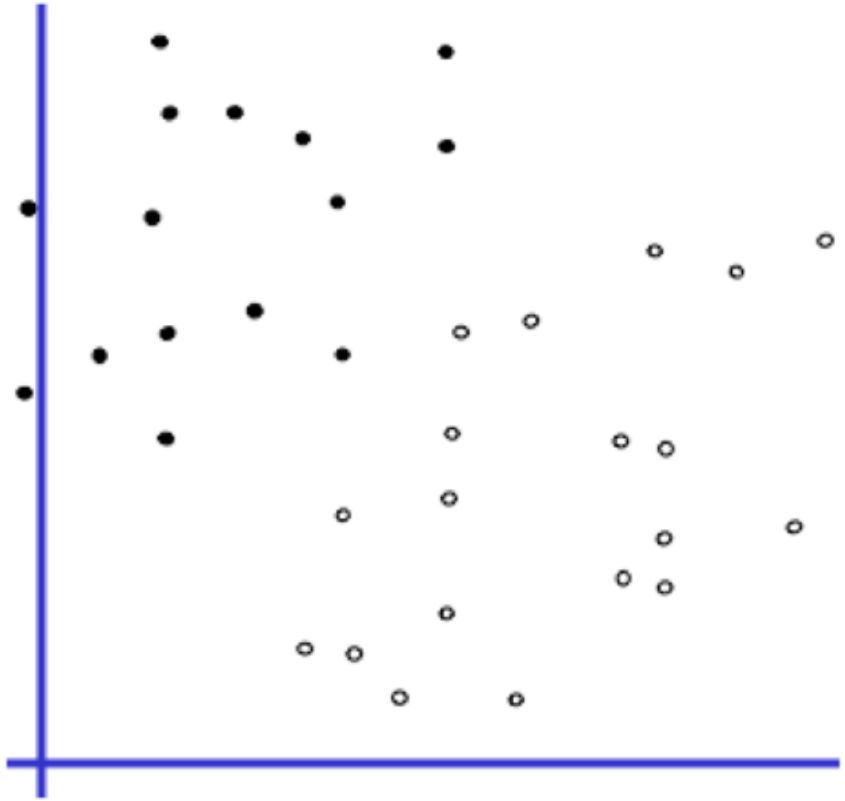
$$
\Delta w_{j0} = \eta \sum_t \left(r_j^t - y_j^t\right)
$$

since, by definition, $x_0^t = 1$.

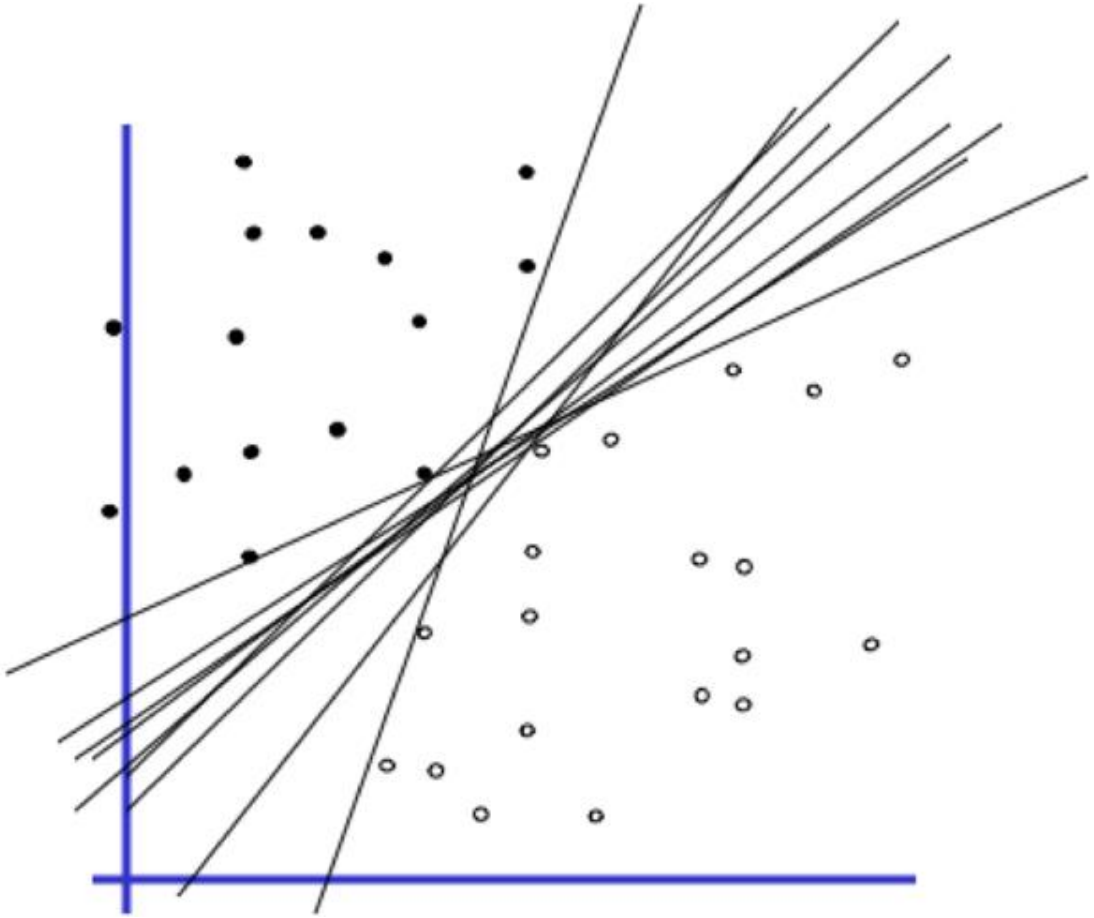
Multi-Class Pseudocode

For $i = 1, \ldots, K$
  For $j = 0, \ldots, d$
    $w_{ij} \leftarrow \text{rand}(-0.01, 0.01)$
Repeat
  For $i = 1, \ldots, K$
    For $j = 0, \ldots, d$
      $\Delta w_{ij} \leftarrow 0$
  For $t = 1, \ldots, N$
    For $i = 1, \ldots, K$
      $o_i \leftarrow 0$
      For $j = 0, \ldots, d$
        $o_i \leftarrow o_i + w_{ij} x_j^t$
    For $i = 1, \ldots, K$
      $y_i \leftarrow \exp(o_i) / \sum_k \exp(o_k)$
    For $i = 1, \ldots, K$
      For $j = 0, \ldots, d$
        $\Delta w_{ij} \leftarrow \Delta w_{ij} + (r_i^t - y_i) x_j^t$
  For $i = 1, \ldots, K$
    For $j = 0, \ldots, d$
      $w_{ij} \leftarrow w_{ij} + \eta \Delta w_{ij}$
Until convergence

A Data Set



A Data Set



Structural Risk Minimization

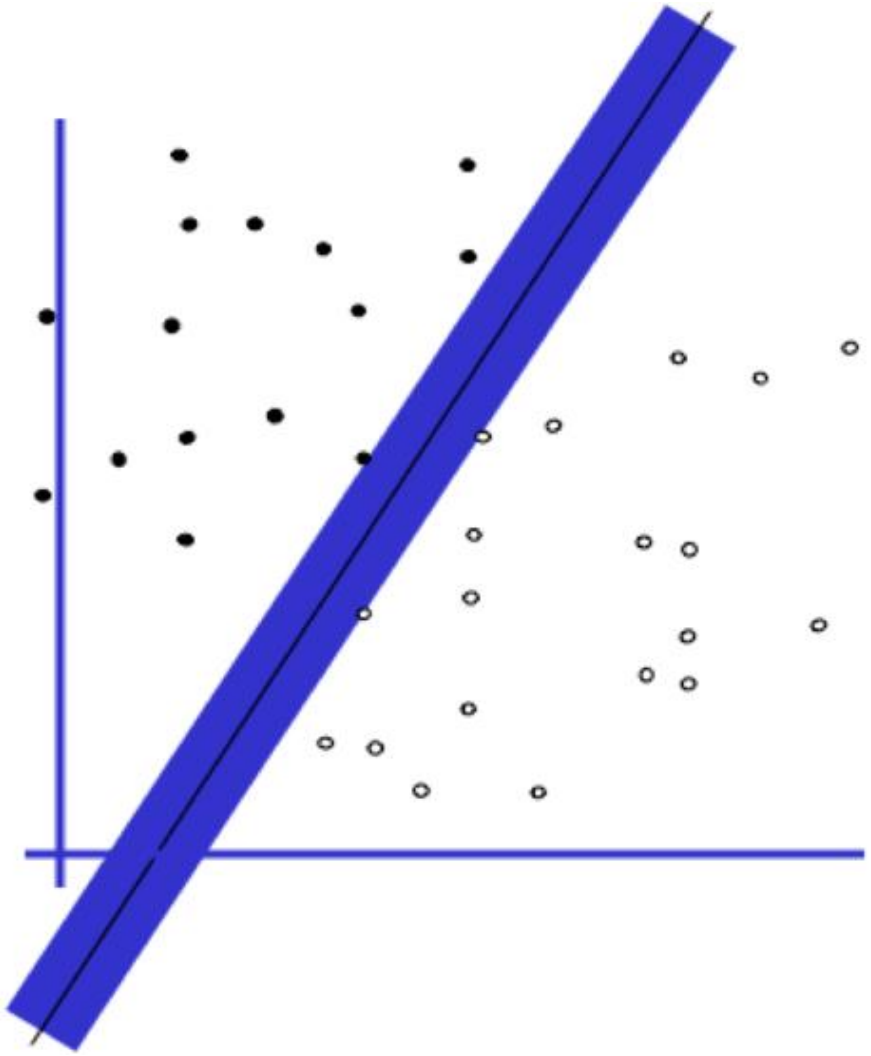
- **Theorem:** The class of optimal linear separators has VC dimension h, bound from above as

$$h \leq \min\left\{\left\lceil\frac{D^2}{\rho^2}\right\rceil, m_0\right\} + 1\,.$$

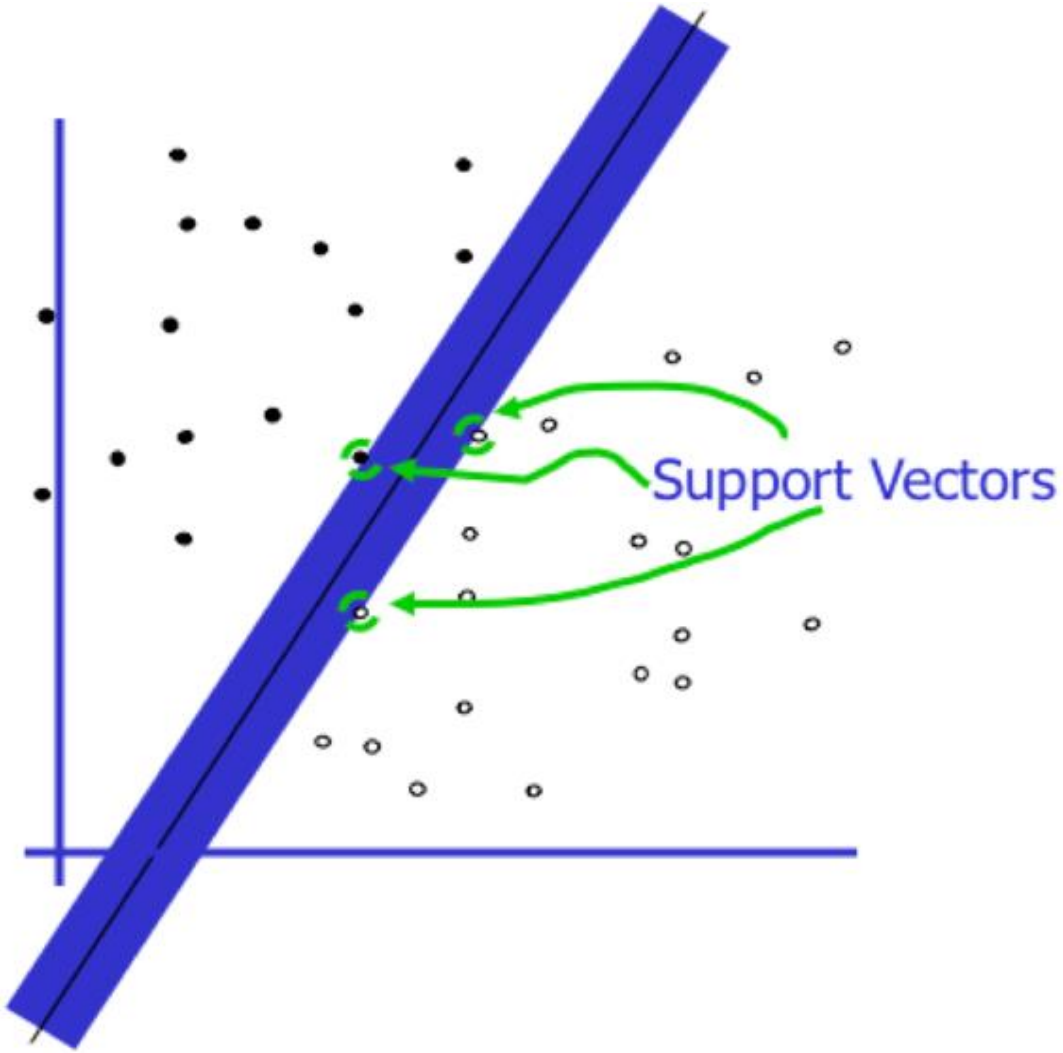
  where ρ is the margin, D is the diameter of the smallest sphere that can enclose all of the training examples, and m_0 is the dimensionality of the input space.

- **Interpretation:** The margin corresponds to the width of the band (or hyper-rectangle) and is the widest the band can be without including one of the data points.

Margin



Support Vectors



Finding the Support Vectors

- Recall a linear discriminant can be represented as $y = \mathbf{w}^\top \mathbf{x} + b$.
- Assuming two-class classification, we can use this as

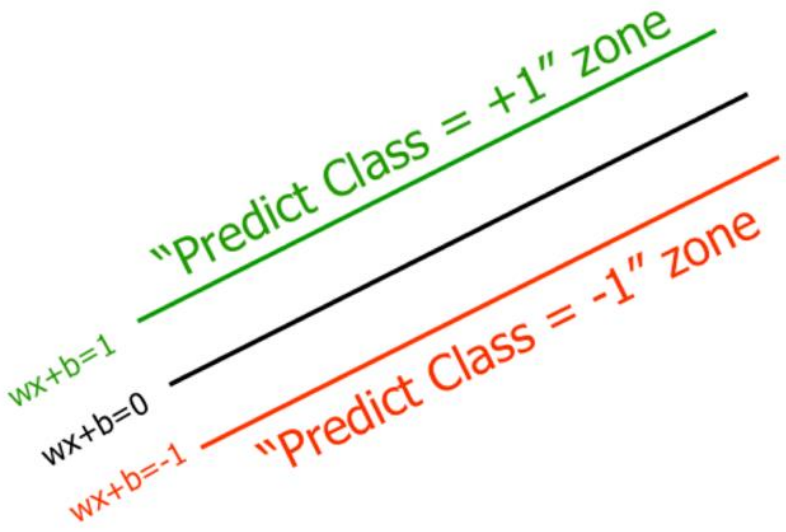
$$class(\mathbf{x}) = \begin{cases} +1 & \mathbf{w}^\top \mathbf{x} + b \geq 0 \\ -1 & \text{otherwise} \end{cases}$$

- Rewrite decision rule as

$$class(\mathbf{x}) = \begin{cases} +1 & \mathbf{w}^\top \mathbf{x} + b \geq +1 \\ -1 & \mathbf{w}^\top \mathbf{x} + b \leq -1 \end{cases}$$

- Margin width

$$\rho = \frac{2}{\sqrt{\mathbf{w}^\top \mathbf{w}}}$$



Maximizing the Margin

- Objective function

$$\max \rho = \max\{2/\sqrt{\mathbf{w}^\top \mathbf{w}}\}$$

- Alternatively

$$\min\{\mathbf{w}^\top \mathbf{w}\}$$

- This is a "quadratic programming" problem with $k = 1, \ldots, n$ constraints.

$$\text{if } y_k = +1 \text{ then } \mathbf{w}^\top \mathbf{x} + b \geq +1$$

$$\text{if } y_k = -1 \text{ then } \mathbf{w}^\top \mathbf{x} + b \leq -1$$

A Limitation

- We are working with a simple linear discriminant.
- This means we are assuming the data is “linearly separable.”
- What if it isn’t?
- Two perspectives:
  - Separability breaks down because of noise.
  - Separability breaks down because classes really aren’t linearly separable.

Handling Noise

There are three possible approaches to handling noise.

1. Minimize $\mathbf{w}^\top \mathbf{w}$ and error simultaneously: A multi-objective optimization problem.
2. Minimize $\mathbf{w}^\top \mathbf{w} + C(\#error_{train})$: Second term is an integer.
3. Minimize $\mathbf{w}^\top \mathbf{w} + C\big(dist(\mathbf{x}_{err}, plane)\big)$: A continuous objective function.

The third method has the nice property that the objectives don't compete directly and can be solved via quadratic programming.

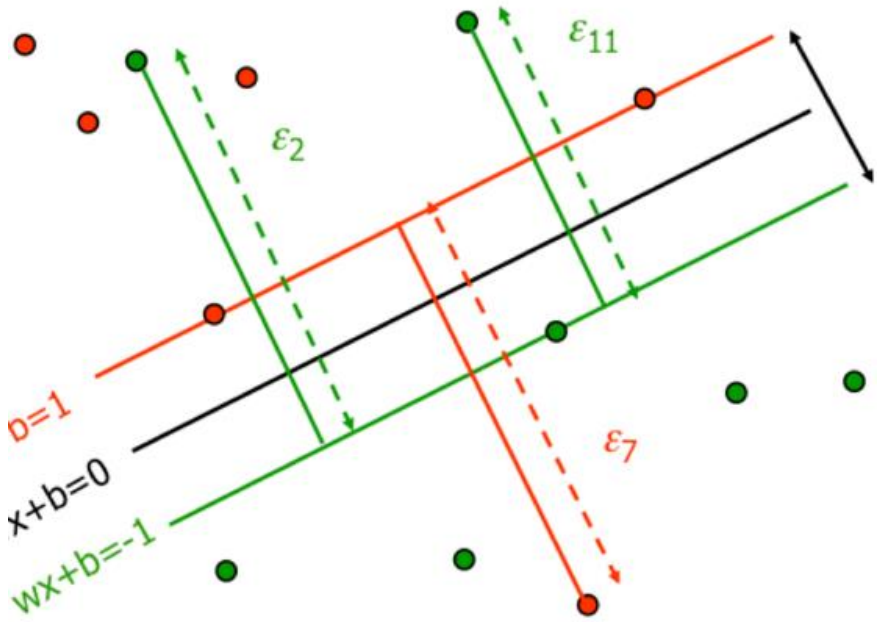
Soft Margin

- Minimizing distance of erroneous points to correct side leads to the ”soft margin.”
- We can then solve the optimization problem by introducing “slack variables.”
- Denote the slack variable for example $\mathbf{x}_k$ as ξ_k.
- New objective function:

$$\text{minimize } \frac{1}{2}\mathbf{w}^\top\mathbf{w} + C\sum_{k=1}^{n}\xi_k$$

subject to

$$\begin{aligned}
\mathbf{w}^\top\mathbf{x} + b &\geq 1 - \xi_k \ (y_k = +1) \\
\mathbf{w}^\top\mathbf{x} + b &\geq -1 + \xi_k \ (y_k = -1) \\
\forall k \ \xi_k &\geq 0
\end{aligned}$$



Lagrangian Method

- Improve efficiency
- Identify support vectors more easily through a dual formulation:

$$\text{maximize} \sum_{k=1}^{n} \alpha_k - \frac{1}{2} \sum_{k=1}^{n} \sum_{\ell=1}^{n} \alpha_k \alpha_\ell Q_{k\ell}$$

where

$$Q_{k\ell} = y_k y_\ell (\mathbf{x}_k^\top \mathbf{x}_\ell)$$

- The corresponding constraints become

$$\forall k \; 0 \leq \alpha_k \leq C$$

$$\sum_{k=1}^{n} \alpha_k y_k = 0$$

- Can solve via quadratic programming or via gradient descent.

Lagrange Multipliers

- We note that the support vectors correspond to the data points where $\alpha_k > 0$.
- We can recover the weights from these support vectors.
- First, let $K = \underset{k}{\operatorname{argmax}}\, \alpha_k$.
- Then

$$b = y_K(1 - \xi_k) - \mathbf{x}_K^\top \mathbf{w}$$

and

$$\mathbf{w} = \sum_{k=1}^{n} \alpha_k y_k \mathbf{x}_k$$

- Finally

$$class(\mathbf{x}') = \operatorname{sgn}(\mathbf{w}^\top \mathbf{x} - b)$$

Support Vector Regression (ϵ-SVR)

- Let $\mathcal{D} = \{(\mathbf{x}_i, y_i\}_{i=1}^n \subset \mathcal{X} \times \mathbb{R}$
- Find function of the form $f(x) = \mathbf{w}^\top \mathbf{x} + b$.
- Minimize $\|\mathbf{w}\|^2 = \mathbf{w}^\top \mathbf{w}$.
- New optimization problem.

$$\text{minimize } \frac{1}{2}\mathbf{w}^\top \mathbf{w}$$

subject to

$$y_i - (\mathbf{w}^\top \mathbf{x}_i + b) \leq \epsilon$$

$$(\mathbf{w}^\top \mathbf{x}_i + b) - y_i \leq \epsilon$$

ϵ-SVR with Soft Margin

Minimize

$$\frac{1}{2}\mathbf{w}^\top\mathbf{w} + C\sum_{k=1}^{n}(\xi_i + \xi_i^*)$$

subject to

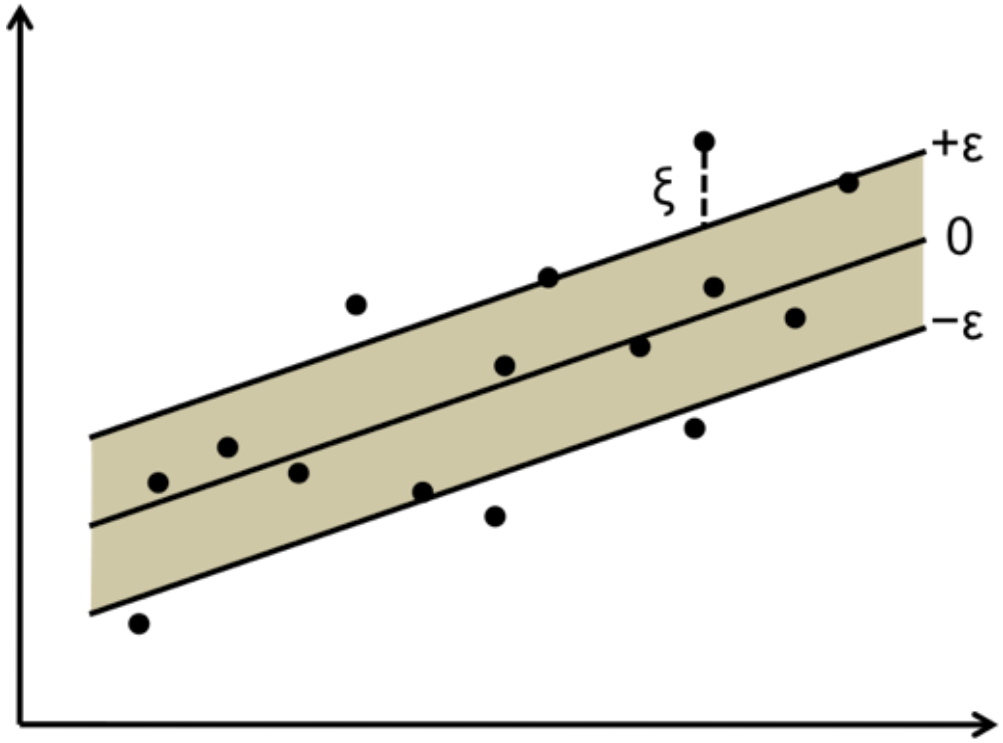
$$y_i - (\mathbf{w}^\top\mathbf{x} + b) \leq \epsilon + \xi_i$$

$$(\mathbf{w}^\top\mathbf{x} + b) - y_i \leq \epsilon + \xi_i^*$$

$$\xi_i, \xi_i^* \geq 0$$

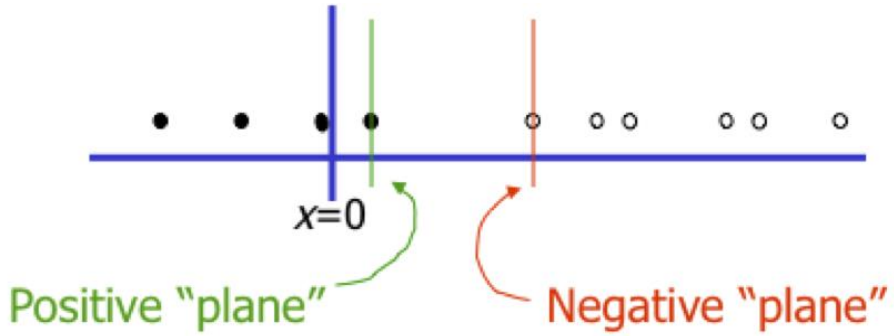
Slack cost

$$|\xi|_\epsilon = \begin{cases} 0 & \text{if } |\xi| \leq \epsilon \\ |\xi| - \epsilon & \text{otherwise} \end{cases}$$

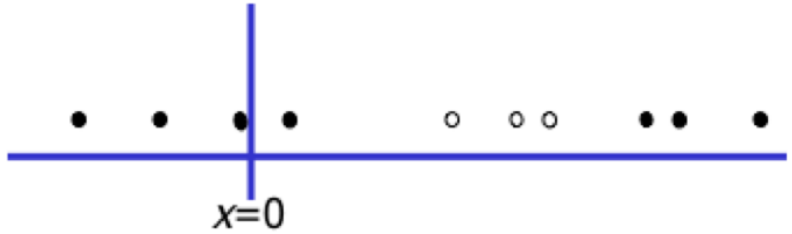


Limitation of the SVM

- SVMs are linear classifiers.
- A linearly separable problem.



- A nonlinearly separable problem.



Cover's Theorem

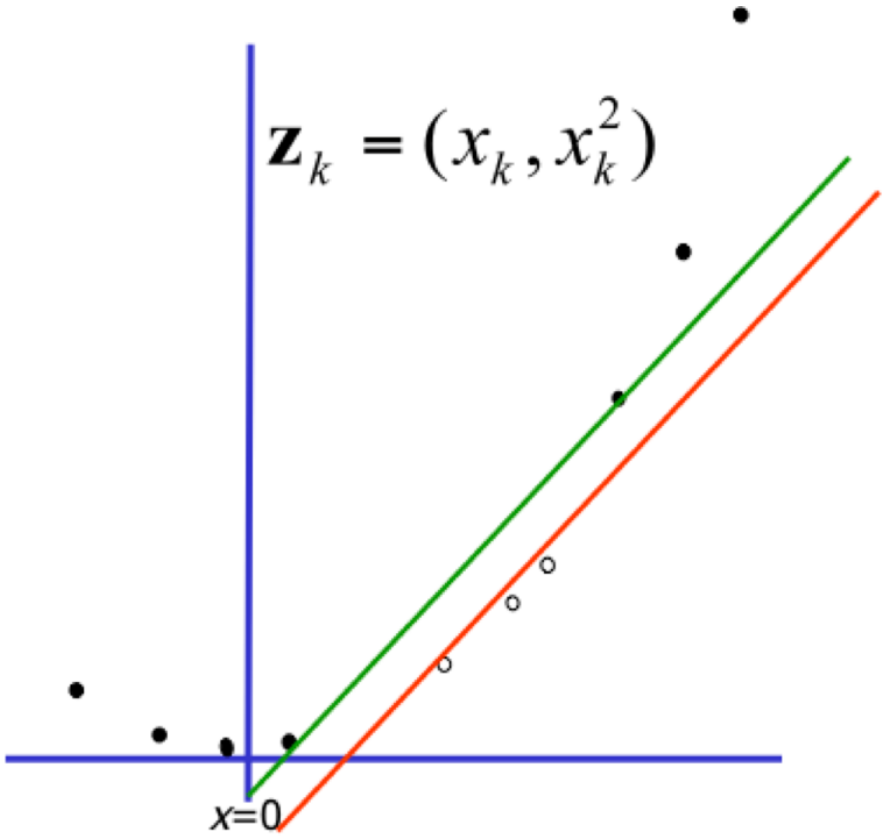
Theorem: There are $C(N, d)$ homogeneous linear dichotomies of N points in general position in Euclidean d-space, where

$$C(N, d) = 2\sum_{k=0}^{d-1}\binom{N-1}{k}.$$

- When $N \leq d + 1$, exponential growth suggests increasing likelihood of linear separability.
- When $N > d + 1$, subexponential growth suggests decreasing likelihood of linear separability.

Theorem (informal): In a classification problem, if the data is cast into a high dimensional space nonlinearly, as long as the resulting space is not densely populated, the data becomes more likely to be linearly separable than in the lower-dimensional space.

Increasing Dimensionality



The Kernel Trick

- Most linear classifiers rely on dot products.
- Often relies on a distance-based calculation.
- A kernel is defined as a function of two vectors:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^\top \mathbf{x}_j$$

- Consider mapping to a higher dimensional space: $\phi: \mathbf{x} \rightarrow \phi(\mathbf{x})$
- Corresponding dot product becomes:

$$K_\phi(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^\top \phi(\mathbf{x}_j)$$

Mercer's Theorem

Definition: A positive, semi-definite matrix **M** is a matrix such that, for all non-zero vectors $\mathbf{z} \in \mathbb{R}^n, \mathbf{x}^\top \mathbf{M}\mathbf{x} \geq 0$. Equivalently, a positive, semi-definite matrix **M** has all non-zero eigenvalues.

Definition: A positive, semi-definite kernel K on a set **S** is a function $K: \mathbf{S} \times \mathbf{S} \rightarrow \mathbb{R}$ such that $\forall n \in \mathbb{N}, \forall x_i \in \mathbf{S}$,

$$\mathbf{C} = \begin{bmatrix} c_{11} & \cdots & c_{1n} \\ \vdots & \ddots & \vdots \\ c_{n1} & \cdots & c_{nn} \end{bmatrix}, c_{ij} = K\left(x_i, x_j\right).$$

Theorem: Any continuous, symmetric, positive, semi-definite kernel function $K(\mathbf{x}, \mathbf{y})$ can be expressed as a dot product in a higher-dimensional space.

Combined with Cover's Theorem, linearize a nonlinear problem, by using a kernel to transform to a higher-dimensional space, thus increasing the probability of the data being linearly separable.

An Example

- Suppose we have vectors of the form $\mathbf{x} = [x_1 \; x_2]^\mathsf{T}$.
- Suppose our kernel is $K(\mathbf{x}_i, \mathbf{x}_j) = (1 + \mathbf{x}_i^\mathsf{T}\mathbf{x}_j)^2$.
- Need to show that $K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^\mathsf{T}\phi(\mathbf{x}_j)$.

$$
\begin{aligned}
K(\mathbf{x}_i, \mathbf{x}_j) &= (1 + \mathbf{x}_i^\mathsf{T}\mathbf{x}_j)^2 \\
&= (1 + 2x_{i1}x_{j1}x_{i2}x_{j2} + x_{i2}^2x_{j2}^2 + 2x_{i1}x_{j1} + 2x_{i2}x_{j2}) \\
&= [1 \quad x_{i1}^2 \quad \sqrt{2}x_{i1}x_{i2} \quad x_{i2}^2 \quad \sqrt{2}x_{i1} \quad \sqrt{2}x_{i2}]^\mathsf{T} \\
&\times \quad [1 \quad x_{j1}^2 \quad \sqrt{2}x_{j1}x_{j2} \quad x_{j2}^2 \quad \sqrt{2}x_{j1} \quad \sqrt{2}x_{j2}]
\end{aligned}
$$

- Then $K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^\mathsf{T}\phi(\mathbf{x}_j)$, where

$$\phi(x) = [1 \quad x_1^2 \quad \sqrt{2}x_1x_2 \quad x_2^2 \quad \sqrt{2}x_1 \quad \sqrt{2}x_2].$$

Standard Kernels

- Polynomial Kernel

$$K_d(\mathbf{x}_i, \mathbf{x}_j) = (1 + \mathbf{x}_i^\top \mathbf{x}_j)^d$$

- Gaussian Kernel

$$K_\mathcal{G}(\mathbf{x}_i, \mathbf{x}_j) = \exp\left(-\frac{(\mathbf{x}_i - \mathbf{x}_j)^2}{2\sigma^2}\right)$$

- Sigmoid Kernel

$$K_\sigma(\mathbf{x}_i, \mathbf{x}_j) = \tanh(\kappa \mathbf{x}_i^\top \mathbf{x}_j - \delta)$$

Kernel-based Objective

- The basic objective function is the same:

$$L_p = \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_t \xi^t$$

- The constraints change:

$$r^t \mathbf{w}^\top \boldsymbol{\phi}(\mathbf{x}^t) \geq 1 - \xi^t$$

- The resulting Lagrangian then becomes:

$$L_p = \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_t \xi^t - \sum_t \alpha^t [r^t \mathbf{w}^\top \boldsymbol{\phi}(\mathbf{x}^t) - 1 + \xi^t] - \sum_t \mu^t \xi^t$$

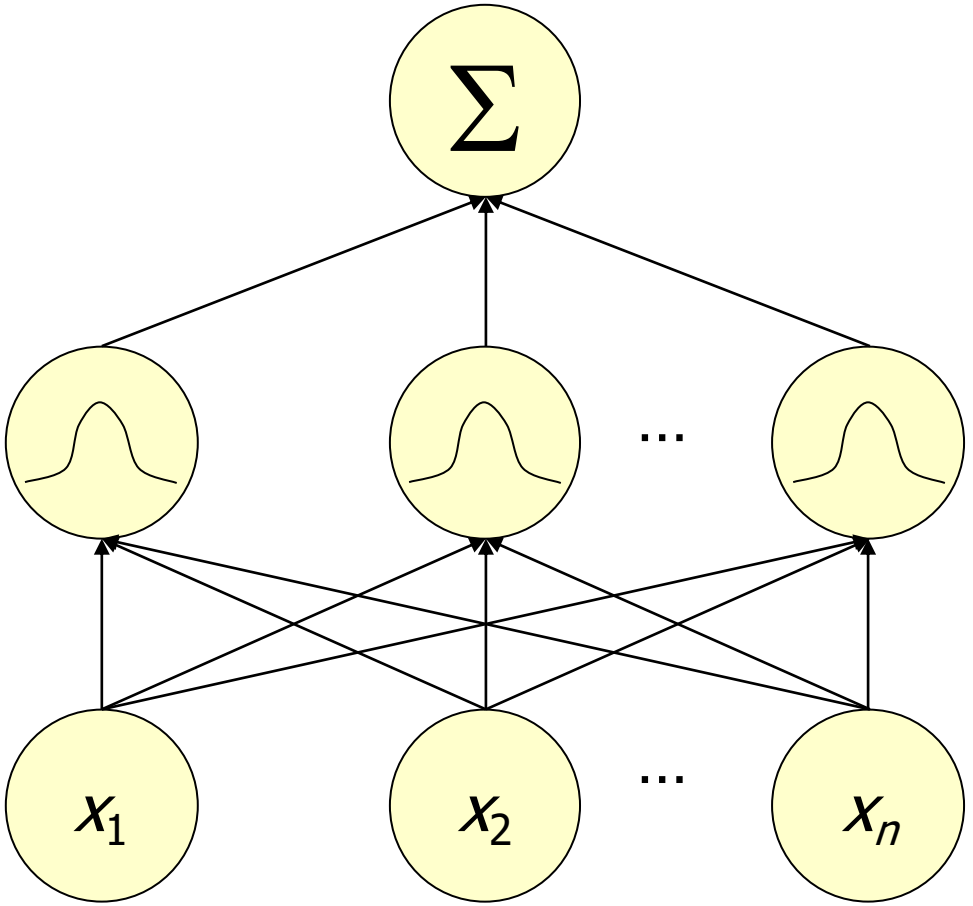
- With the dual:

$$L_d = \sum_t \alpha^t - \frac{1}{2}\sum_t \sum_s \alpha^t \alpha^s r^t r^s \boldsymbol{\phi}(\mathbf{x}^t)^\top \boldsymbol{\phi}(\mathbf{x}^s)$$

- Shown with kernel:

$$L_d = \sum_t \alpha^t - \frac{1}{2}\sum_t \sum_s \alpha^t \alpha^s r^t r^s K(\mathbf{x}^t, \mathbf{x}^s)\,.$$

Radial Basis Function Network



The Radial Basis Function

- A symmetric function with “radial” symmetry.
- “On center, off surround” property of the visual cortex.

$$f(\mathbf{x}) = G(\|\mathbf{x} - \boldsymbol{\mu}\|)$$

- Gaussian Kernel

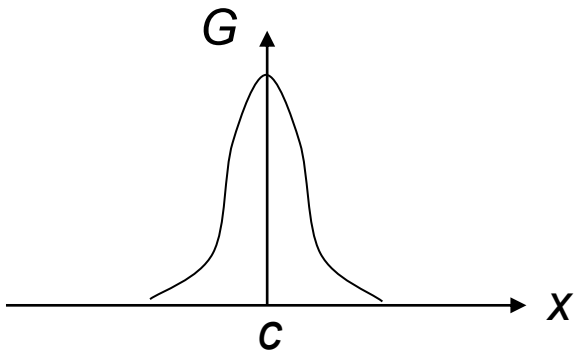
$$G(y) = \exp\left(-\frac{y^2}{\sigma^2}\right), \qquad \text{where } \sigma \text{ is spread}$$

- Usually, this is represented as

$$K(\mathbf{x}, \boldsymbol{\mu}) = \exp(-\gamma\|\mathbf{x} - \boldsymbol{\mu}\|^2) = \exp\left(-\frac{\|\mathbf{x} - \boldsymbol{\mu}\|^2}{2\sigma^2}\right), \qquad \text{where } \gamma = \frac{1}{2\sigma^2}$$

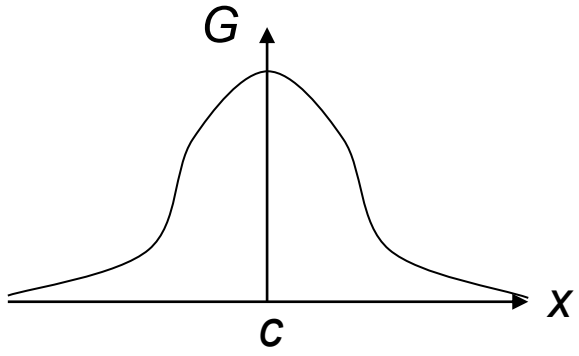
Spread/Selectivity

Small spread



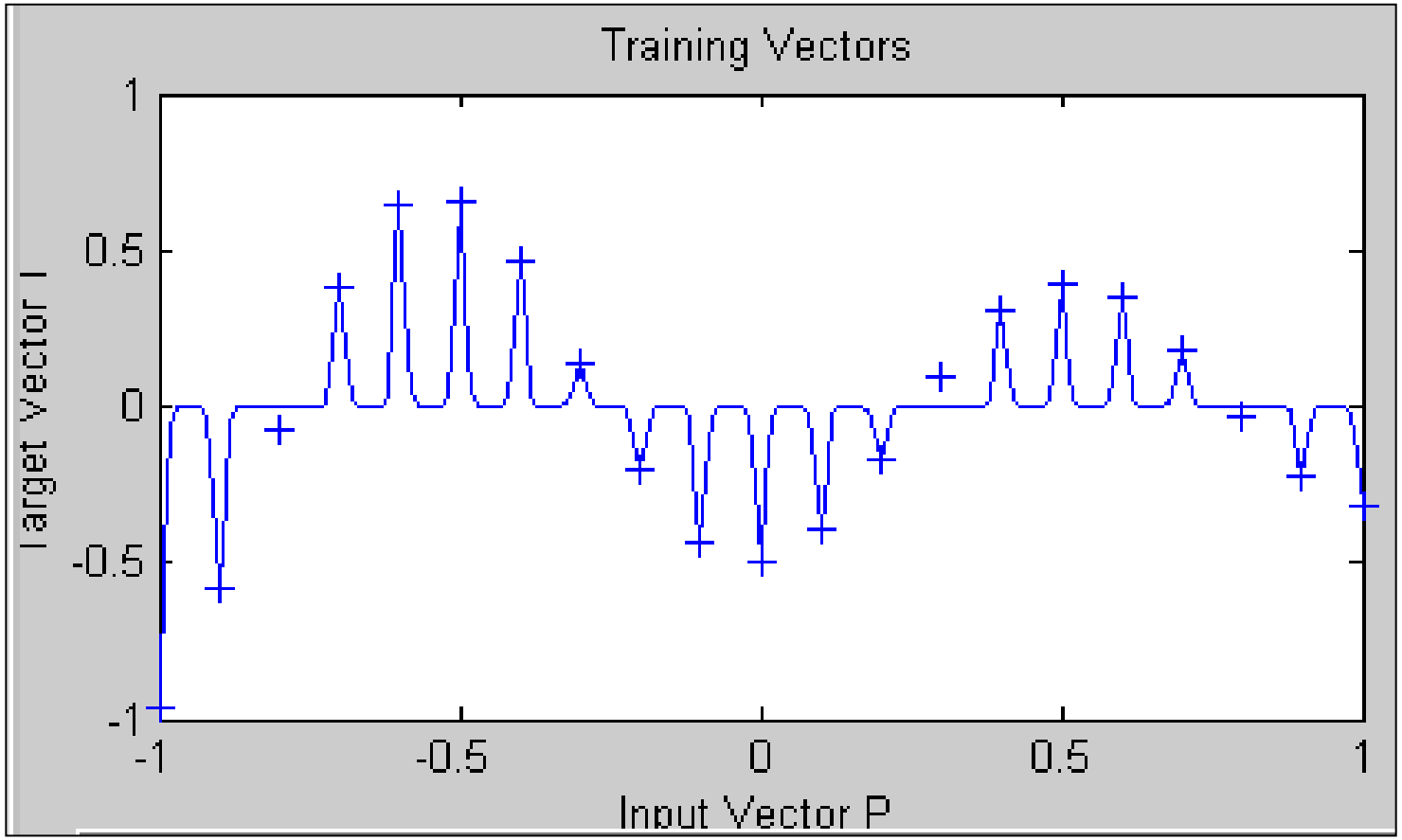
selective

Large spread

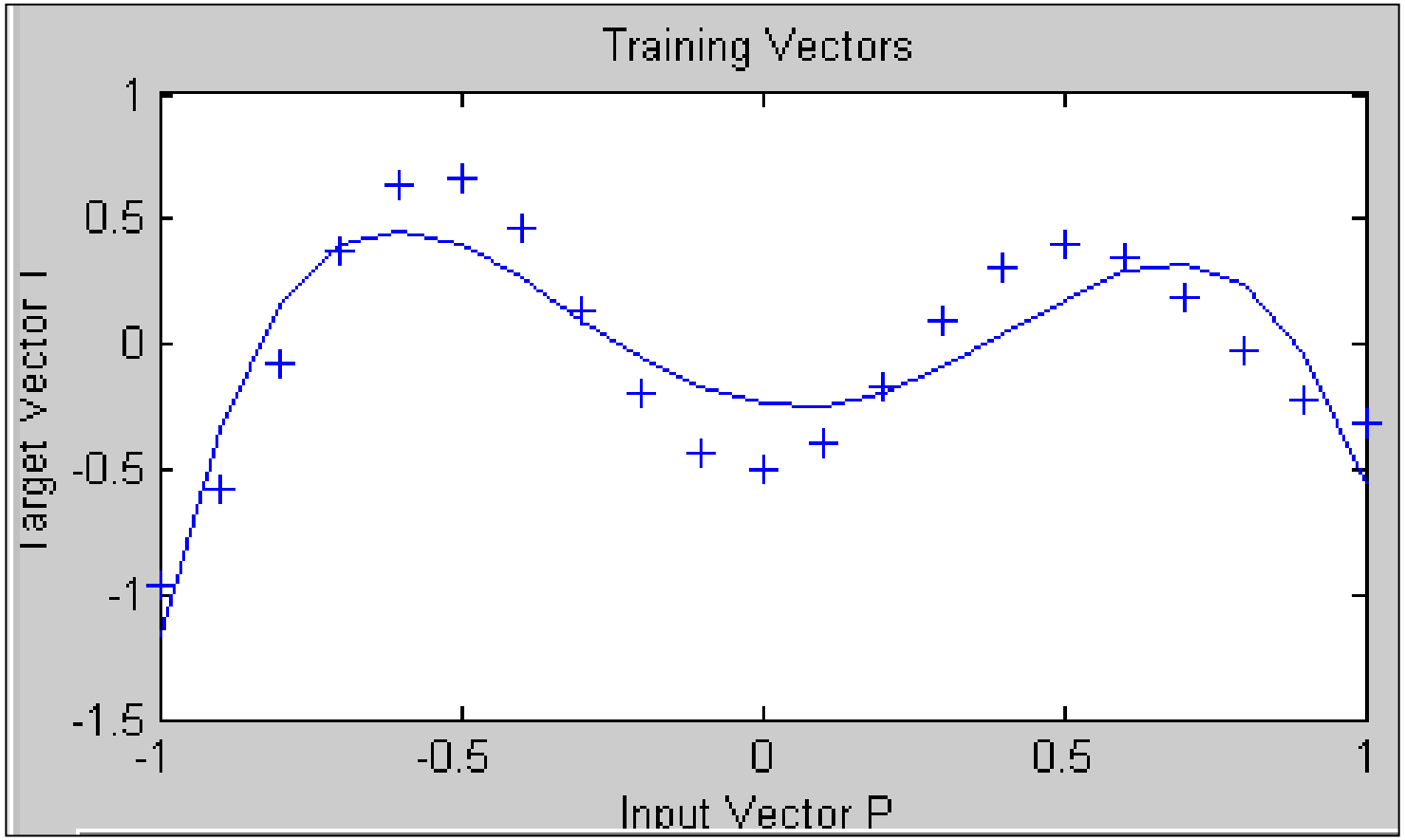


not selective

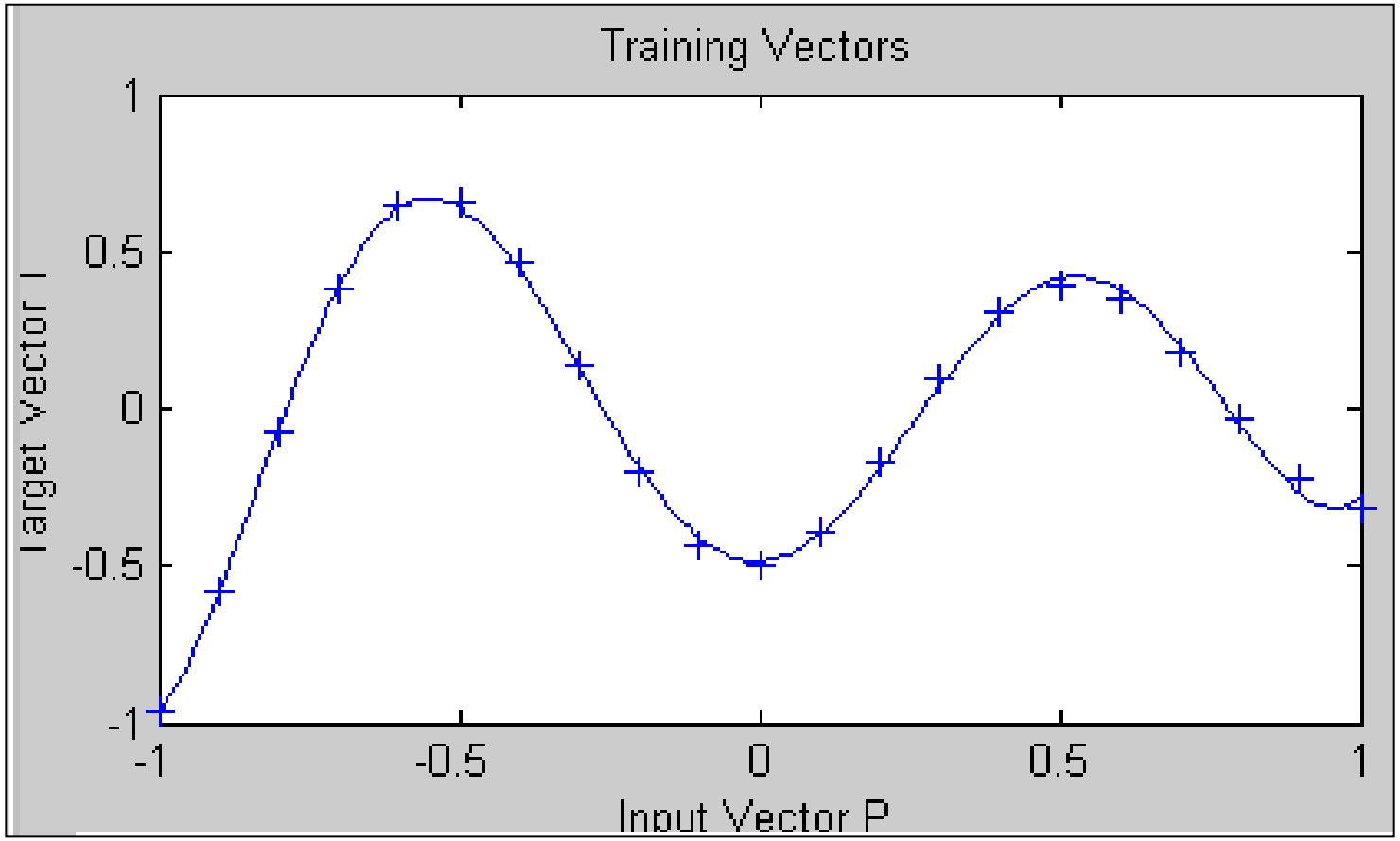
Small Spread



Large Spread



Ideal Spread



RBF Network

- Bridges the gap between
  - Neural networks (e.g., feedforward networks)
  - Kernel machines (e.g., SVMs)
  - Nonparametric/instance-based learning (e.g., nearest neighbor)
- Setting the spread
  - Tune a global parameter
  - Clustering the data and estimating cluster distributions
  - Train via gradient descent
- Linear combiner – gradient descent

Network Architecture

Three layers

1. Input Layer: One node per feature
2. Hidden Layer: $k \leq n$ nodes applying RBF kernel
3. Output layer:
   - Classification: One class per node
   - Regression: Single node

- The RBF kernel: $K_j(\mathbf{x}, \boldsymbol{\mu}_j) = \exp\left(-\frac{1}{2\sigma_j^2}\|\mathbf{x} - \boldsymbol{\mu}_j\|^2\right)$
- Then $f(x) \approx \sum_{j=1}^{k} w_j K_j(\mathbf{x}, \boldsymbol{\mu}_j)$

Training RBF Parameters

- Apply *k*-means clustering
- Associate one hidden node per example
- The center $\boldsymbol{\mu}_j$ is the centroid of the cluster or the individual example $\mathbf{x}_j$
- Once again, this yields

$$K_j(\mathbf{x}, \boldsymbol{\mu}_j) = \exp\left(-\frac{1}{2\sigma_j^2}\|\mathbf{x} - \boldsymbol{\mu}_j\|^2\right)$$

Training RBF Parameters

- Apply gradient descent to learn the centers μ_{hj} and spreads σ_h.
- We can find the derivatives with respect to the centers and spreads using the chain rule and a process that looks like back propagation (see later in course).

$$\Delta\mu_{hj} = \eta \sum_t \left[\sum_i \left(r_i^t - y_i^t \right) w_{ih} \right] p_h^t \frac{\left(x_j^t - \mu_{hj} \right)}{\sigma_h^2}$$

$$\Delta\sigma_h = \eta \sum_t \left[\sum_i \left(r_i^t - y_i^t \right) w_{ih} \right] p_h^t \frac{\left\| \mathbf{x}^t - \boldsymbol{\mu}_h \right\|_2^2}{\sigma_h^3}$$

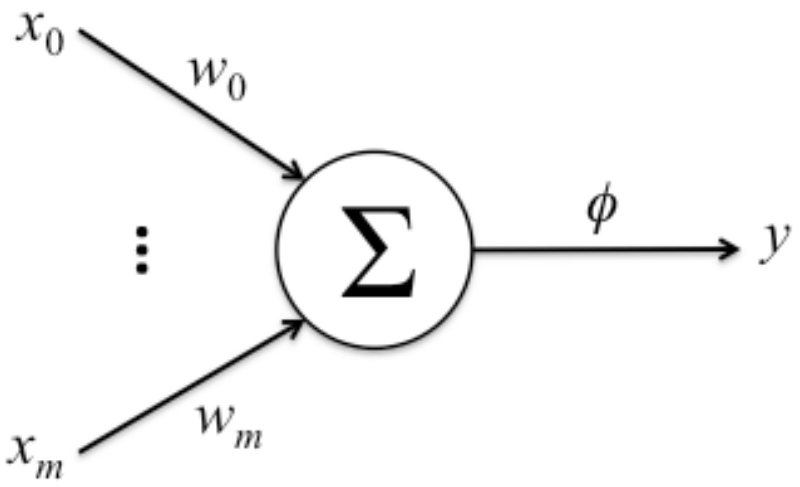
RBF Output

- Loss function:
  - For classification: Cross entropy
    - $\mathcal{L} = r^t \log y^t + (1 - r^t) \log(1 - y^t)$
  - For regression: Squared error
    - $\mathcal{L} = (r^t - y^t)^2$
- Update rule is based on $\partial \mathcal{L} / \partial w$.
  - For classification:
    - $\Delta w = \eta(r^t - y^t)p_h$
  - For regression:
    - $\Delta \mathrm{w} = \eta(r^t - y^t)p_h$

The Perceptron – Early Linear Neural Network

- Recall notion of function approximation:
  - $f: \mathbf{X} \subset \mathbb{R}^m \rightarrow \mathbf{Y} \subset \mathbb{R}^n$
  - $\mathcal{D} = \{(\mathbf{x}^t, \mathbf{y}^t)\}_{t=1}^{|\mathcal{D}|}$
  - $\mathbf{y}^t = f(\mathbf{x}^t)$ indicates the "target"
- The model representation is a network (artificial neural network):
  - Neurons
  - Nodes
- Activation function

$$y_k = \phi\left(\sum_{i=1}^{m} w_{ki} x_i\right)$$



The Loss Function

- The perceptron is generally used for classification.
- Even so, the loss function used is mean squared error.
- The network encodes a function $\hat{f}_{\mathbf{w}}(\mathbf{x}) \approx f(\mathbf{x})$.
- Minimize

$$MSE(\mathbf{w}) = \frac{1}{|\mathcal{D}|} \sum_{\mathbf{x} \in \mathcal{D}} \left(f(\mathbf{x}) - \hat{f}_{\mathbf{w}}(\mathbf{x}) \right)^2$$

- We find $\mathbf{w}^* = \text{argmin}_{\mathbf{w}} \; MSE(\mathbf{w})$.

The Perceptron Network

- Two layers of neurons/nodes.
  - Distributes the inputs
  - Apply activation function to weighted sum

$$\phi(y) = \phi\left(\sum_{i=1}^{d} w_i x_i - \theta\right) = \mathrm{sgn}\left(\sum_{i=1}^{d} w_i x_i - \theta\right)$$

where

$$\mathrm{sgn}(x) = \begin{cases} +1 & x > 0 \\ -1 & x \leq 0 \end{cases}$$

- Activation is a step function (aka, hard limiter or Heaviside function)

Perceptron Training

- Let
  - $\mathbf{w} = [w_1, \ldots, w_d]^\top$
  - $\mathbf{x} = [x_1, \ldots, x_d]^\top$
  - $z = \text{sgn}(\mathbf{w}^\top \mathbf{x})$
  - $\mathbf{F} = \mathbf{F}^+ \cup \mathbf{F}^-$
- $\mathbf{F}$ is the training set
  - $\mathbf{F}^+$ denotes positive examples
  - $\mathbf{F}^-$ denotes negative examples

1. Choose initial weight vector $\mathbf{w}$ at random
2. Choose some example $\mathbf{x} \in \mathbf{F}$
3. Consider the $class(\mathbf{x})$
   - If $\mathbf{x} \in \mathbf{F}^+$ and $\text{sgn}(\mathbf{w}^\top \mathbf{x}) \leq 0$, then $\mathbf{w} \leftarrow \mathbf{w} + \mathbf{x}$
   - If $\mathbf{x} \in \mathbf{F}^-$ and $\text{sgn}(\mathbf{w}^\top \mathbf{x}) > 0$, then $\mathbf{w} \leftarrow \mathbf{w} - \mathbf{x}$
   - Otherwise, do nothing (class is correct)
4. Repeat steps 2 and 3 until convergence

Example – Logical OR

x_1	x_2	$x_1 \vee x_2$
0	0	0
0	1	1
1	0	1
1	1	1

Train

- Initial weight vector $\mathbf{w} = [0\ 0]^T$
- For (0,0), $\operatorname{sgn} \mathbf{w} = 0$: No change in weights
- For (0,1), $\operatorname{sgn} \mathbf{w} = 0$: Update $\mathbf{w} = [0\ 1]^T$
- For (1,0), $\operatorname{sgn} \mathbf{w} = 0$: Update $\mathbf{w} = [1\ 1]^T$
- For (1,1), $\operatorname{sgn} \mathbf{w} = 1$: No change in weights

Try again

- For (0,0), $\operatorname{sgn} \mathbf{w} = 0$: No change in weights
- For (0,1), $\operatorname{sgn} \mathbf{w} = 1$: No change in weights
- For (1,0), $\operatorname{sgn} \mathbf{w} = 1$: No change in weights
- For (1,1), $\operatorname{sgn} \mathbf{w} = 1$: No change in weights

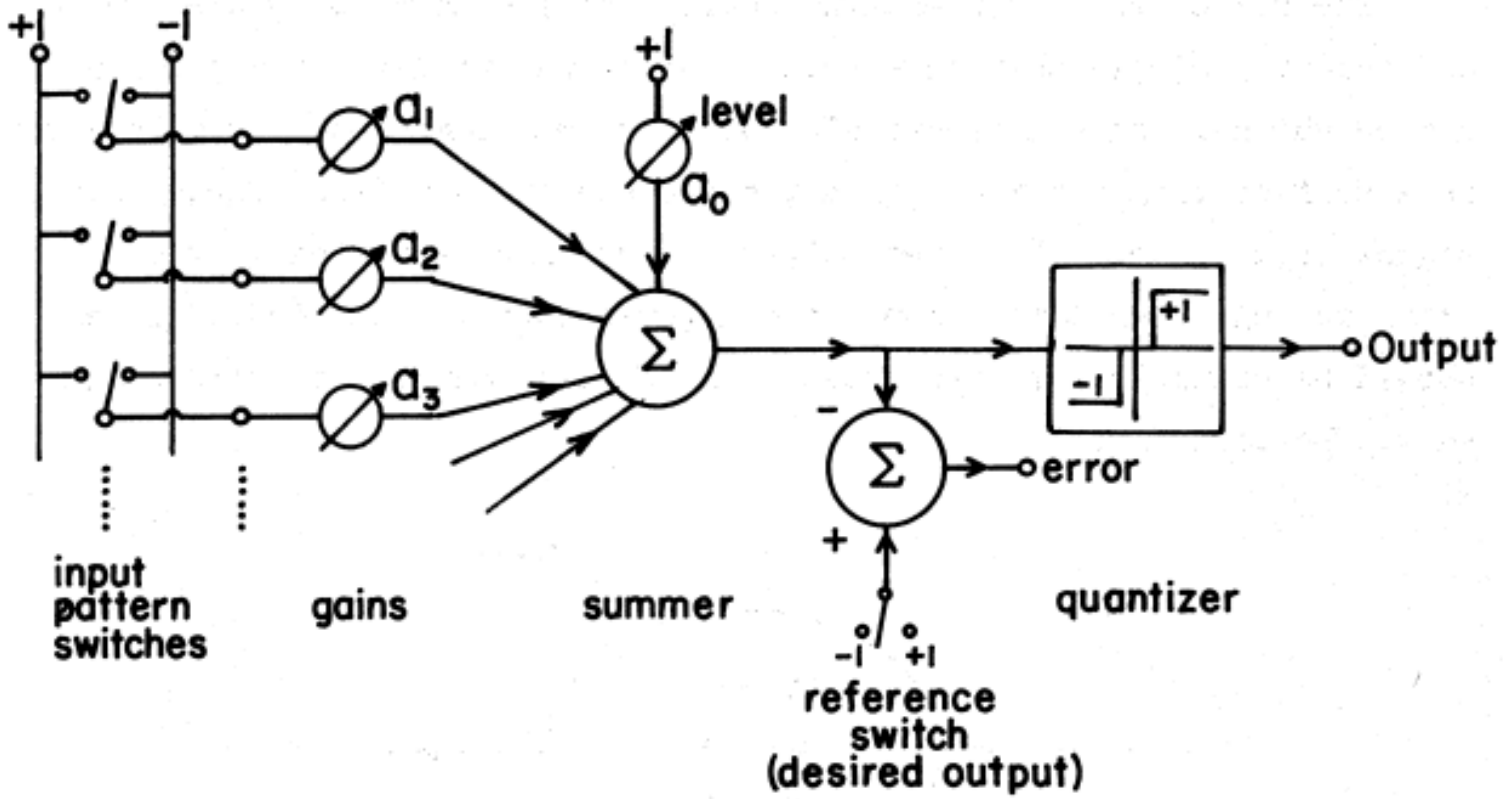
Adaline

- What is Adaline?
  - **Ada**ptive **Li**near **Ne**uron
  - **Ada**ptive **Lin**ear **E**lement
- Similar to the perceptron,
  - Let $\mathbf{w} = [w_1, \ldots, w_d]^\top$ be a weight vector
  - Let $\mathbf{x} = [x_1, \ldots, x_d]^\top$ be an input vector
  - Let θ be a threshold
- Compute

$$y = \sum_{i=1}^{d} w_i x_i - \theta$$

- Note, $x_0 = -1$ and $w_0 = \theta$

Adaline



B. Widrow, *An Adaptive "Adaline" Neuron Using Chemical "Memristors"* Technical Report No. 1553-2, Stanford University, October 17, 1960.

Adaline Training

- Let
  - y: network output $= \hat{f}_{\mathbf{w}}(\mathbf{x})$
  - d: desired output $= f(\mathbf{x})$
- Minimize squared error: $Err = (d - y)^2$
- Apply a learning rate $\eta \in (0,1)$
- Then the weight update rule is

$$\mathbf{w}_{t+1} \leftarrow \mathbf{w}_t + \eta(d - y)\mathbf{x}$$

Widrow-Hoff Rule

- The function we are learning: $\hat{f}_{\mathbf{w}}(\mathbf{x}) = \mathbf{w}^{\top}\mathbf{x}$
- Mean squared error:

$$MSE(\mathbf{w}) = \frac{1}{|\mathcal{D}|}\sum_{i=1}^{|\mathcal{D}|}(f(\mathbf{x}_i) - \mathbf{w}^{\top}\mathbf{x}_i)^2 = \frac{1}{|\mathcal{D}|}\sum_{i=1}^{|\mathcal{D}|}(d_i - y_i)^2$$

- Calculating the gradient:

$$\nabla_{\mathbf{w}} MSE(\mathbf{w}) = -\frac{2}{|\mathcal{D}|}\sum_{i=1}^{|\mathcal{D}|}(f(\mathbf{x}_i) - \mathbf{w}^{\top}\mathbf{x}_i)\mathbf{x}_i$$

- Weight update:

$$\mathbf{w}_{t+1} \leftarrow \mathbf{w}_t + \eta\nabla_{\mathbf{w}} MSE(\mathbf{w}_t) = \mathbf{w}_t + \eta\frac{1}{|\mathcal{D}|}\sum_{i=1}^{|\mathcal{D}|}(f(\mathbf{x}_i) - (\mathbf{w}_t^{\top}\mathbf{x}_i))\mathbf{x}_i$$

Adaline vs Perceptron

Adaline:

- Batch updating
- $\mathbf{w}_{t+1} \leftarrow \mathbf{w}_t + \eta(f(\mathbf{x}_i) - \mathbf{w}_t^{\top}\mathbf{x}_i)\mathbf{x}_i$

Perceptron:

- Incremental updating:
- $\mathbf{w}_{t+1} \leftarrow \mathbf{w}_t \pm \mathbf{x}$

When correct

- Positive: $\mathbf{w}_{t+1} \leftarrow \mathbf{w}_t + \eta\big(+1 - (+1)\big)\mathbf{x}_i = \mathbf{w}_t$
- Negative: $\mathbf{w}_{t+1} \leftarrow \mathbf{w}_t + \eta\big(-1 - (-1)\big)\mathbf{x}_i = \mathbf{w}_t$

When incorrect

- Should be Positive: $\mathbf{w}_{t+1} \leftarrow \mathbf{w}_t + \frac{1}{2}\big(+1 - (-1)\big)\mathbf{x}_i = \mathbf{w}_t + \mathbf{x}_i$
- Should be Negative: $\mathbf{w}_{t+1} \leftarrow \mathbf{w}_t + \frac{1}{2}\big(-1 - (+1)\big)\mathbf{x}_i = \mathbf{w}_t - \mathbf{x}_i$

Perceptron Training (Recap)

- Recall training set $\mathbf{F} = \mathbf{F}^{+} \cup \mathbf{F}^{-}$
- Start by considering $\mathbf{x} \in \mathbf{F}^{-}$: $\mathbf{x} \leftarrow -\mathbf{x}$

1. Choose initial weight vector $\mathbf{w}$ at random
2. Choose some example $\mathbf{x} \in \mathbf{F}$
3. Consider the $class(\mathbf{x})$
   - If $\text{sgn}(\mathbf{w}^{\top}\mathbf{x}) \leq 0$, then $\mathbf{w} \leftarrow \mathbf{w} + \mathbf{x}$
   - Otherwise, do nothing (class is correct)
4. Repeat steps 2 and 3 until convergence

Perceptron Convergence Theorem

Theorem: Let $\mathbf{F}$ be a set of unit-length vectors, and let $\mathbf{w}$ be an initial weight vector. If there exists a unit vector $\mathbf{w}'$ and a number $\delta > 0$ such that $\mathbf{w}'^{\mathsf{T}}\mathbf{x} > \delta$ for all $\mathbf{x} \in \mathbf{F}$, then the perceptron learning rule will update $\mathbf{w}$ only a finite number of times to find such a $\mathbf{w}'$.

Convergence Proof

- Recall that $\mathbf{w}'$ is an arbitrary weight vector.
- Since it's a unit vector, $\|\mathbf{w}'\| = 1$.
- Define $g(\mathbf{w}) = \frac{\mathbf{w}'^{\mathsf{T}}\mathbf{w}}{\|\mathbf{w}\|} \leq 1$.
- Assume, wlg, we have misclassified a negative example.
- Consider, first, the numerator of $g(\mathbf{w})$.

$$\begin{aligned} \mathbf{w}'^{\mathsf{T}}\mathbf{w}_{t+1} &= \mathbf{w}'^{\mathsf{T}}(\mathbf{w}_t + \mathbf{x}) \\ &= \mathbf{w}'^{\mathsf{T}}\mathbf{w}_t + \mathbf{w}'^{\mathsf{T}}\mathbf{x} \end{aligned}$$

- Then $\mathbf{w}'^{\mathsf{T}}\mathbf{x} > \delta \Longrightarrow \mathbf{w}'^{\mathsf{T}}\mathbf{w}_{t+1} > \mathbf{w}'^{\mathsf{T}}\mathbf{w}_t + \delta$
- Then for n updates, $\mathbf{w}'^{\mathsf{T}}\mathbf{w}_{t+1} > n\delta$.

Convergence Proof

- Now consider the denominator of $g(\mathbf{w})$.
- We only update when $\mathbf{w}^\top \mathbf{x} \leq 0$.
- Thus

$$\begin{aligned} \|\mathbf{w}_{t+1}\|^2 &= \mathbf{w}_{t+1}^\top \mathbf{w}_{t+1} \\ &= (\mathbf{w}_t + \mathbf{x})^\top (\mathbf{w}_t + \mathbf{x}) \\ &= \|\mathbf{w}_t\|^2 + 2\mathbf{w}_t^\top \mathbf{x} + \|\mathbf{x}\|^2. \end{aligned}$$

- Since $\mathbf{w}_t^\top \mathbf{x} \leq 0$ and $\|\mathbf{x}\|^2 \leq 1$, we must also have that $\|\mathbf{w}_{t+1}\|^2 \leq \|\mathbf{w}_t\|^2 + 1$.
- Again, after n updates, $\|\mathbf{w}_n\|^2 \leq n \implies \frac{1}{\|\mathbf{w}_n\|^2} \geq \frac{1}{n}$.

Convergence Proof

- Combining the results for the numerator and the denominator:

$$\begin{aligned} g(\mathbf{w}_n) &= \frac{\mathbf{w}'^{\mathsf{T}}\mathbf{w}_n}{\|\mathbf{w}_n\|} \\ &> \frac{n\delta}{\sqrt{n}} \\ &= \delta\sqrt{n}. \end{aligned}$$

- Since $g(\mathbf{w}_n) \leq 1$, this requires that $\delta\sqrt{n} \leq 1$.
- This gives us

$$n \leq \frac{1}{\delta^2}.$$

Q.E.D.